

LAB_7

More Recursive Algorithms and Structures

Session Objectives

By the end of this session, you will be able to:

1. Split problems recursively using divide & conquer
2. Draw fractals with recursive functions
3. Build recursive spatial structures (like quadtrees)
4. Generate natural-looking patterns procedurally

Core Exercises

Exercise 1: Divide & Conquer – Spatial Splitting

Objective: Implement recursive functions that split a 2D space into smaller regions.

Requirements:

1. Implement `split_region(x, y, width, height, min_size)`
 - `x, y` are the top-left corner coordinates of a rectangular region (`width, height`)
 - If region is smaller than `min_size`, stop
 - Else, split into 4 equal quadrants
 - Recursively call `split_region` on each quadrant
2. Implement `count_points_in_region(points, region)`
 - `points` can be generated randomly
 - Count how many points fall inside a given `region`
 - Use with the split function to find dense areas
3. Implement `find_dense_regions(points, min_size, density_threshold)`
 - Recursively split space
 - Return only regions where point density > threshold

Example:

- Level 0: [100 points in 100x100]
- Level 1: [25][30][20][25] points in 50x50 each
- Level 2: [8][7][6][9] ... in 25x25 each
- Stop when region < 10x10

Complexity Analysis Questions:

1. If you have N points and split until size = 1x1, how many recursive calls?
2. What happens if points are all clustered in one corner?

Exercise 2: Fractal Drawing – Recursive Shapes

Objective: Draw self-similar shapes using recursion.

Requirements:

1. Implement `draw_sierpinski(canvas, x, y, size, depth)`
 - If `depth == 0`, draw a triangle
 - Else, draw 3 smaller Sierpinski triangles inside
2. Implement `draw_tree(canvas, x, y, length, angle, depth)`
 - If `depth == 0`, draw a line (leaf)
 - Else, draw a line, then draw 2 smaller branches at ± 30 degrees
3. Implement `fractal_dimension(fractal_image, box_sizes)`
 - For each box size, count how many boxes contain part of the fractal
 - Plot $\log(\text{count})$ vs $\log(1/\text{size}) \rightarrow \text{slope} = \text{fractal dimension}$

Complexity Analysis questions:

1. For Sierpinski depth 5, how many small triangles are drawn?
2. What is the fractal dimension of a straight line? Of a filled square?

Exercise 3: Procedural Generation – Recursive Patterns

Objective: Generate random-looking natural patterns using recursion.

Requirements:

1. Implement `midpoint_displacement(x1, y1, x2, y2, roughness, depth)`
 - Find midpoint = $(x1+x2)/2, (y1+y2)/2$
 - Add random offset = $\text{roughness} \times \text{random}(-1,1)$
 - Recursively process left half and right half
 - Stop when `depth = 0`
2. Implement `generate_terrain(width, height, roughness, depth)`
 - Start with 4 corners at height 0
 - Apply diamond-square algorithm recursively
 - Return 2D array of heights
3. Implement `detect_artifacts(terrain_grid, threshold)`
 - Find grid cells where height changes too abruptly
 - Return list of suspicious coordinates

Complexity Analysis Questions:

1. For a 1024-pixel line with depth 10, how many midpoints are calculated?
2. What happens if `roughness = 0`? If `roughness = 2`?

Final Integration Question

Build a "Recursive Pattern Generator" that combines all three exercises.

What to build:

A program that generates and analyzes recursive patterns.

Features needed:

Feature	What it does	Exercise
Split view	Show space divided into regions	Exercise 1
Draw fractal	Display Sierpinski or tree	Exercise 2
Generate terrain	Create random landscape	Exercise 3
Measure dimension	Calculate fractal dimension	Exercise 2
Find anomalies	Detect unnatural patterns	Exercise 3

Simple scenario:

1. User clicks "Generate Terrain" → creates a landscape
2. System measures fractal dimension → shows $D = 2.3$
3. If D is too low (<1.8) or too high (>2.5), show warning
4. User clicks "Split View" → sees quadtree regions overlaid
5. User clicks "Find Artifacts" → highlights unnatural straight lines

Questions to answer:

1. Recursion depth: For a 512×512 grid, what's a good max depth? Why?
2. Performance: The system should respond in <1 second. How would you make it fast? (Hint: stop recursion early, use small test images)
3. Storage: If you save fractal dimension every second for 1 hour, how many data points? Is that too many?
4. Edge cases: What happens if you try to draw Sierpinski with depth = 100? How would you prevent crashes?

General Instructions

Repository Setup

Create branch: LAB_07_BasicsOfGraphs

File Structure:

Follow the same structure of LAB_06

Git Requirements:

Same as for LAB_06

Documentation & Notation

At the End of the Session	Before Friday Midnight of the Same Week
Note: 5 • Each Team (of number BB) Upload at the end of the session a very short PDF document and name it TBB_LAB7_AES.pdf, containing for each exercise: the algorithm in Pseudo-Code (according to the rules explained in lecture 1) – Brief answers of the questions, • You can implement your algorithms to make sure that are working, • NO AI-generated content (will be checked for authenticity), • NO code snippets.	Note: 15 • You finish all the implementation of all exercises, • You finalize the previous report and give it a new name TBB_LAB7_BFM.pdf and upload it before Friday midnight of the same week, containing for each exercise: same as previous + more complexity analysis and reflection + Test set for edge cases of your Algorithms' implementation, • AI-generated content is not recommended, • A link to the Team's Repository in GitHub.